INFOTACT SOLUTIONS & CO.

Infotact Technical Internship Program: Software Engineering Project Specifications and Evaluation Criteria

The following document establishes the technical requirements, architectural directives, and sprint-based development roadmaps for three distinct software engineering projects. Designed for execution within a fast-paced technology environment in Bengaluru, Karnataka, these specifications serve as comprehensive blueprints for Generative AI engineering interns entering the infotact startup ecosystem. The documentation provides structured frameworks intended to guide the development of Large Language Model (LLM) applications, Retrieval-Augmented Generation (RAG) pipelines, and Agentic AI systems across three high-growth industry niches: Education, Professional Services, and Media & Entertainment.

The primary directive of this documentation is to standardize the AI engineering process, ensuring that all deliverables align with modern GenAI architecture patterns, rigorous version control protocols, and robust prompt security standards. Consequently, engineering interns must operate at a professional level, demonstrating proficiency in vector databases, context window management, API integrations, and Continuous Integration.

This document outlines the requisite tasks, technological paradigms, and evaluation criteria for successful internship completion. It is imperative to note that the assessment of these projects relies heavily on the transparency and consistency of the development lifecycle. **The evaluation will only be done if the intern is having all 4 weeks of github commits and contributions.**

Project 1: Education & EdTech - Generative AI Tutor & Adaptive Learning Platform

Executive Problem Statement

Traditional education systems frequently rely on a one-size-fits-all approach, which fails to accommodate the unique pacing and comprehension levels of individual students. Educators struggle to provide 24/7 personalized support, leaving students stuck on complex concepts. While standard search engines provide information, they lack the pedagogical structure required to teach a concept effectively without simply giving away the answer.

The objective of this project is to build an intelligent AI Tutoring System utilizing Generative AI and Retrieval-Augmented Generation (RAG). The platform will ingest specific course curriculums and use LLMs to provide personalized, conversational tutoring, generate dynamic quizzes, and adapt its teaching style based on the student's real-time performance.

Business Objectives and Key Performance Indicators

The strategic goal is to simulate a 1-on-1 personalized tutoring experience that improves student retention and comprehension. The system must rely strictly on the provided educational material, decisively preventing the AI from "hallucinating" or providing factually incorrect answers from outside the syllabus.

Success will be quantified by the retrieval accuracy of the RAG pipeline. Key metrics include Context Relevance (measuring if the retrieved chunks actually contain the answer) and Answer Faithfulness (ensuring the LLM's response is strictly derived from the retrieved context). System latency should be kept under 2 seconds per conversational turn to maintain engagement.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Student	Instant, interactive help with coursework and adaptive practice.	Chats with the AI tutor to clarify doubts, requests topic summaries, and takes dynamically generated flashcards/quizzes.
Educator/Admin	Easy upload of course materials and visibility into student struggles.	Uploads PDF/Word syllabuses, monitors a dashboard to see which topics students are asking the AI about the most, and identifies curriculum gaps.

Minimum Viable Product Specifications

The foundational feature is the Document Ingestion Pipeline. The system must parse educational PDFs, split the text into semantically meaningful chunks, convert them into vector embeddings, and store them in a Vector Database.

The core feature is the Conversational Tutor Interface. The intern must build an API that takes a student's question, performs a similarity search in the vector database to retrieve the relevant syllabus context, and passes both the context and question to an LLM. The system must maintain conversational memory to answer follow-up questions accurately.

Architectural Directives and Technology Stack

Component	Technology	Architectural Rationale
Backend Framework	Python, FastAPI	FastAPI is the dominant, high-performance choice for AI startups, perfectly suited for building asynchronous LLM APIs.
LLM Orchestration	LangChain / LlamaIndex	Provides essential wrappers for chunking documents, managing conversational memory, and chaining prompts.
Vector Database	Pinecone or ChromaDB	Highly optimized for storing vector embeddings and executing rapid semantic similarity searches.
Frontend Application	React / Streamlit	Streamlit allows for rapid prototyping of chat interfaces, while React is used for production-ready UIs.

Four-Week Engineering Roadmap

Week 1: Ingestion Pipeline and Vector Database Setup

Interns must initialize the GitHub repository and set up the FastAPI project. The primary task is building the data ingestion script. The intern must utilize libraries like `PyPDF2` to read uploaded course materials, use LangChain's `RecursiveCharacterTextSplitter` to chunk the text, generate embeddings using OpenAI's embedding models, and upsert these vectors into Pinecone or ChromaDB.

Week 2: Retrieval-Augmented Generation (RAG) Implementation

Week two focuses on the core AI logic. The intern will build the retrieval endpoint. When a user submits a query, the system must embed the query, fetch the top-K most relevant chunks from the vector database, and construct a strict prompt template (e.g., "You are an expert tutor. Use ONLY the following context to answer the student's question..."). The endpoint must return the LLM's response alongside the source document citations.

Week 3: Conversational Memory and Adaptive Quizzes

The third week introduces complexity. The intern must implement session-based memory (e.g., `ConversationBufferMemory`) so the AI remembers previous questions in the chat. Furthermore, the intern will create a new endpoint that utilizes LLM structured output (JSON generation) to automatically generate 5 multiple-choice questions based on a specific topic requested by the user.

Week 4: UI Integration, Streaming, and Evaluation

The final sprint involves wiring the FastAPI backend to a chat-based frontend. The intern must implement Server-Sent Events (SSE) to stream the AI's response token-by-token for a better user experience. The intern will finalize the repository, ensuring all API keys are secured via `.env` files, and provide a detailed `README.md` explaining the RAG architecture.

Project 2: Professional Services - Agentic AI Document Review System

Executive Problem Statement

In professional services (Law, Consulting, Product Management), professionals spend an exorbitant amount of time drafting, reviewing, and verifying massive documents such as Product Requirement Documents (PRDs), legal contracts, and compliance memos. Manual review is prone to human error, inconsistencies, and formatting deviations, causing severe operational bottlenecks.

The objective of this project is to build an Agentic AI Document Workflow. Using an autonomous multi-agent architecture, the system will accept a rough draft or bullet points, automatically generate a highly structured professional document (like a PRD), and utilize a secondary "Reviewer AI" to critique and refine the document against predefined company standards.

Business Objectives and Key Performance Indicators

The strategic goal is to reduce document turnaround time from hours to minutes while maintaining absolute structural consistency.

Success will be measured by the system's ability to consistently output valid, deeply structured Markdown or JSON documents without dropping required sections. The project will be evaluated on the successful implementation of "Agentic Routing"—the ability of the AI to self-correct and loop back if the internal reviewer agent detects missing information.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Product Manager / Consultant	Rapid transformation of rough meeting notes into polished, actionable documents.	Inputs a raw text prompt or meeting transcript. The system outputs a fully structured PRD containing user stories, edge cases, and tech specs.
Compliance/QA Reviewer	Ensuring documents adhere strictly to company templates and lack material errors.	Acts as the "Human-in-the-loop," receiving flagged issues from the AI reviewer before giving final approval.

Minimum Viable Product Specifications

The foundational feature is the Multi-Agent Orchestrator. Instead of a single LLM call, the system must utilize distinct AI personas: a "Writer Agent" responsible for drafting the content, and a "Critic Agent" responsible for evaluating the draft against a strict rubric.

The core deliverable requires state management between these agents. The system must process an input, generate a draft, pass it to the Critic, and if the Critic finds missing elements (e.g., "You forgot to include success metrics"), the Writer must automatically revise the document before presenting it to the human user.

Architectural Directives and Technology Stack

Component	Technology	Architectural Rationale
Backend API	Python, FastAPI	Required for exposing the complex agent workflows as consumable REST endpoints.

Agent Framework	LangGraph or CrewAI	LangGraph provides robust state management and graph-based routing for multi-agent cyclic workflows.
LLM Provider	Anthropic Claude 3.5 or GPT-4o	Claude models excel exceptionally at long-form writing, coding, and document analysis tasks.

Four-Week Engineering Roadmap

Week 1: Prompt Engineering and System Prompts

The project initiates with defining the system architecture. The intern will design the strict system prompts for the two core agents: The Writer and The Critic. The intern must experiment with few-shot prompting, providing the LLM with examples of high-quality PRDs or Memos to establish the desired tone and structure. Basic API calls will be tested in Jupyter Notebooks.

Week 2: Implementing the Multi-Agent Graph

Week two centers on orchestration. The intern must utilize LangGraph (or CrewAI) to build the workflow state. The intern will code nodes (functions) for generating the document, evaluating the document, and a conditional edge (routing logic) that loops the process back to the Writer if the Critic's score falls below a certain threshold.

Week 3: Structured Output and Tool Calling

The third week transitions to robust data extraction. The intern will implement Function Calling/Tool Use to force the LLM to output its final review in a strict JSON format (e.g., `{"status": "approved", "feedback": [...]}`). This ensures the backend application can predictably parse the AI's output without regex errors.

Week 4: Human-in-the-Loop and API Wrapping

The final week is dedicated to deployment and safety. The intern must wrap the LangGraph workflow in a FastAPI backend. They will implement a "Human-in-the-Loop" pause, allowing the workflow to suspend execution and wait for human approval via an API call before finalizing the document. The GitHub repository must contain detailed documentation on how the agent graph is constructed.

Project 3: Media & Entertainment - Multi-Modal AI Content Marketing Engine

Executive Problem Statement

Digital marketing and media companies face immense pressure to produce high volumes of content across various platforms (blogs, Instagram, Twitter, newsletters). Currently, content creators must jump between disparate tools to write copy, generate images, and format posts, resulting in fragmented workflows and inconsistent brand messaging.

This project requires the development of an integrated, Multi-Modal AI Content Engine. The application will allow users to input a single product description or campaign idea, and the AI will simultaneously generate platform-optimized copywriting, complementary AI-generated imagery, and relevant SEO metadata.

Business Objectives and Key Performance Indicators

The strategic vision is to act as a digital content multiplier, reducing the friction of campaign creation.

From an engineering standpoint, the system's success relies on asynchronous task management and multimodal API orchestration. Generating high-resolution images and complex text simultaneously takes time; therefore, the application must handle long-running background tasks without timing out or freezing the client's browser.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Marketing Campaign Manager	Rapid generation of cohesive assets for a new product launch.	Inputs a brief ("Launch campaign for eco-friendly sneakers"). The system returns a blog post, 3 tweet variants, and 2 AI-generated promotional images.
Social Media Executive	Platform-specific formatting and instant visual assets.	Approves, edits, or regenerates specific assets through the dashboard before exporting them for publishing.

Minimum Viable Product Specifications

The application foundation is the Campaign Generator API. The intern must integrate multiple AI models: a text-generation model (e.g., GPT-4o) for the copywriting, and an image-generation model (e.g., DALL-E 3 or Stable Diffusion via Replicate) for the visual assets.

The core technical deliverable is the Asynchronous Task Queue. Because generating multiple AI assets can take 10-30 seconds, the intern must implement a message broker architecture (like Celery + Redis) to process the AI generation requests in the background, allowing the frontend to poll for task completion.

Architectural Directives and Technology Stack

Component	Technology	Architectural Rationale
Backend Services	Node.js / Express or Python / FastAPI	Both are highly capable of orchestrating external API calls. Python is preferred if utilizing custom diffusion models.
Task Queue	Celery + Redis (Python) or BullMQ (Node)	Essential for offloading long-running AI generation tasks from the main API thread to prevent server timeouts.
AI APIs	OpenAI (GPT & DALL-E)	Provides state-of-the-art multimodal generation capabilities through a unified developer platform.

Four-Week Engineering Roadmap

Week 1: Multimodal API Integration

The project begins with establishing the backend repository. The intern must securely configure their API keys using environment variables. The primary task is writing the core service functions to interact with the LLM for text generation and the Image model for visual generation, experimenting with prompt engineering to ensure the image style matches the requested text tone.

Week 2: Asynchronous Task Queue Setup

Week two focuses on backend scalability. The intern will configure Redis and set up a task queue worker (Celery or BullMQ). When a user requests a campaign, the main API must immediately return a `task_id` and push the heavy AI generation logic to the background worker. An additional endpoint must be created to check the status of the `task_id`.

Week 3: Content Structuring and Parallel Execution

The third week tackles optimization. The intern must structure the text prompts to ask the LLM for JSON outputs, separating the generated Blog Post, Tweets, and SEO tags into distinct data fields. The intern must also implement parallel execution—triggering the image generation and text generation simultaneously to cut the overall user wait time in half.

Week 4: Dashboard Integration and Final Polish

The final sprint involves wiring the asynchronous backend to a frontend dashboard (e.g., React or Next.js). The frontend must implement a loading state or progress bar that polls the backend using the `task_id` until the generated assets are ready to be displayed. The repository must be finalized, ensuring the commit history reflects a steady progression of solving the asynchronous architecture challenges over the entire four weeks.

Universal Engineering and Version Control Guidelines

To simulate a rigorous, enterprise-grade AI research and development environment indicative of top-tier deep-tech companies in Bengaluru, strict adherence to modern MLOps practices and version control is an absolute requirement. The core tenet of this internship program is continuous, transparent, and high-quality contribution. AI Engineering is an iterative, experimental process, and your version control history must reflect this.

Mandatory Evaluation Protocol

The evaluation of the intern's performance and the final project submission is governed by a strict, non-negotiable directive regarding version control consistency.

The evaluation will only be done if the intern is having all 4 weeks of github commits and contributions.

A project submitted with a compressed commit history—for example, uploading the entire codebase in a massive, monolithic push during the final week or on the final day—will result in immediate disqualification. Professional engineering is a continuous process of iteration, and the version control history must accurately reflect daily progress, prompt refinements, architectural pivots, and debugging efforts.

GitHub Operational Standards and Security Practices

To pass the technical evaluation, interns must internalize and strictly adhere to the following Git and GitHub operational standards.

1. Commit Frequency, Granularity, and Semantic Messaging

Commits act as the immutable ledger of an engineer's thought process. Interns are expected to commit code frequently, breaking down complex features into logical, highly isolated units of work. A minimum of three to five meaningful commits per active development day is the expected baseline. Monolithic commits containing thousands of lines of code spanning multiple disparate features are strictly prohibited.

Commit messages must communicate the exact intent and context of the change. Interns must utilize structured prefixes such as `feat:`, `fix:`, `prompt-tuning:`, or `refactor:` to maintain a highly readable repository history (e.g., `prompt-tuning: updated system prompt to output strict JSON format`).

2. Branching Strategy and Pull Request (PR) Etiquette

Direct commits to the `main` or `master` branch are entirely forbidden. All new development must occur on isolated feature branches. Code is merged into the main branch exclusively via Pull Requests. Each PR must contain a comprehensive description outlining the specific problem solved, the prompt engineering approach taken, and testing instructions.

3. Continuous Integration and Cryptographic Security (CI/CD)

Security hygiene is paramount, especially in Generative AI where leaked API keys can result in massive financial loss. Hardcoding OpenAI, Anthropic, or Pinecone API keys within the Python source code is grounds for immediate failure. All sensitive data must be injected via secure Environment Variables (`.env` files) and the `.env` file must be strictly excluded from the repository using `.gitignore`.

By strictly enforcing these comprehensive specifications, the program ensures that deliverables transcend mere API wrappers, transforming into robust, scalable, and highly maintainable Generative AI applications indicative of elite industry standards.